AGRUPAMENTO DE ESCOLAS GABRIEL PEREIRA - ÉVORA

Programação e Sistemas de Informação

MÓDULO 9

INTRODUÇÃO À PROGRAMAÇÃO ORIENTADA POR OBJETOS

CURSO PROFISSIONAL

TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

DISCIPLINA DE PROGRAMAÇÃO E SISTEMAS DE INFORMAÇÃO

Programação Orientada a Objetos (POO)



- Não substitui a programação estruturada;
- Evolução das técnicas de programação a usar e também usa programação estruturada;
- Permite criar bibliotecas para partilhar e reutilizar o código;
- Reduz o tempo de desenvolvimento;
- Simplifica o processo de manutenção das aplicações.

Conceitos da POO



- Como funciona?
  - Aproxima o mundo lógico da programação ao mundo real.
  - Tudo o que existe são objetos.
  - Os objetos são criados através das suas características e funcionalidades.
  - As características são as suas propriedades ou atributos.
  - As funcionalidades são os seus métodos.

Conceitos da POO



- Conceitos fundamentais da POO
  - Abstração
  - Encapsulamento
  - Herança
  - Polimorfismo

Conceitos da POO



- Abstração
  - Representação de objetos reais;
  - Imaginar o que esse objeto vai fazer no sistema a desenvolver;
  - Dar-lhe identidade (nome adequado e único no sistema);
  - Identificar propriedades;
  - Definir as ações que irá executar (métodos);

Conceitos da POO



🐕 Objeto	💼 Propriedades
Cão	Tamanho, Raça, Idade

⚙️ Métodos

Latir(), Dormir(), Comer()

Conceitos da POO



- Encapsulamento
  - Permite adicionar segurança;
  - "Esconde" as propriedades (*private*);
  - A alteração do valor das propriedades é feita através de métodos especiais (*get* e *set*);
  - Evita acesso direto à propriedade adicionando uma camada de proteção.

Conceitos da POO



- Herança
  - Os objetos podem herdar atributos e métodos.
- Exemplo
  - O objeto *cão* herda o que está definido para o objeto *animal*.

Conceitos da POO

- Polimorfismo
  - Diretamente ligado com a herança, permite que o mesmo método comportamentos distintos em objetos que tenham o mesmo “pai”;
  - O método pode ter várias (poli) formas (morfismo) ou versões, para ser utilizado em diferentes contextos.

Animal (Pai)

Define o método genérico Dormir()

Cão (Filho)

Redefine Dormir() — deita-se e dorme profundamente

Morcego (Filho)

Redefine Dormir() — pendura-se de cabeça para baixo

Conceitos da POO

- Como começar?



Conceitos da POO



- Classe
  - "Molde" com base no qual se podem criar objetos
  - Unidade de código a partir da qual se podem criar objectos
  - Propriedades (ou atributos) + métodos (ou ações)
- Objeto
  - Instância de uma classe (instanciação – criação de um objeto)
  - Unidades de código

Classes



- Diagrama de classes
  - Para representar as classes necessárias no sistema;
  - Representam-se por retângulos
    - Nome no topo;
    - Atributos na secção intermedia, definidos no seguinte formato:

      nome: tipo de dado ou **nome: tipo de dado = valor**
    - As operações (métodos) a implementar na secção em baixo definidos no seguinte formato:

      Método(): tipo de retorno

Diagrama de Classes



Esta classe é:

- identificada pelo nome Triângulo
- tem como atributos base e altura
- executa a operação Calc_Area(), a partir dos atributos

Diagrama de Classes



ContaBancaria
numero: texto saldo: número real dataAbertura: data
criar(): void bloquear(): void desbloquear(): void creditar(): void debitar(): void

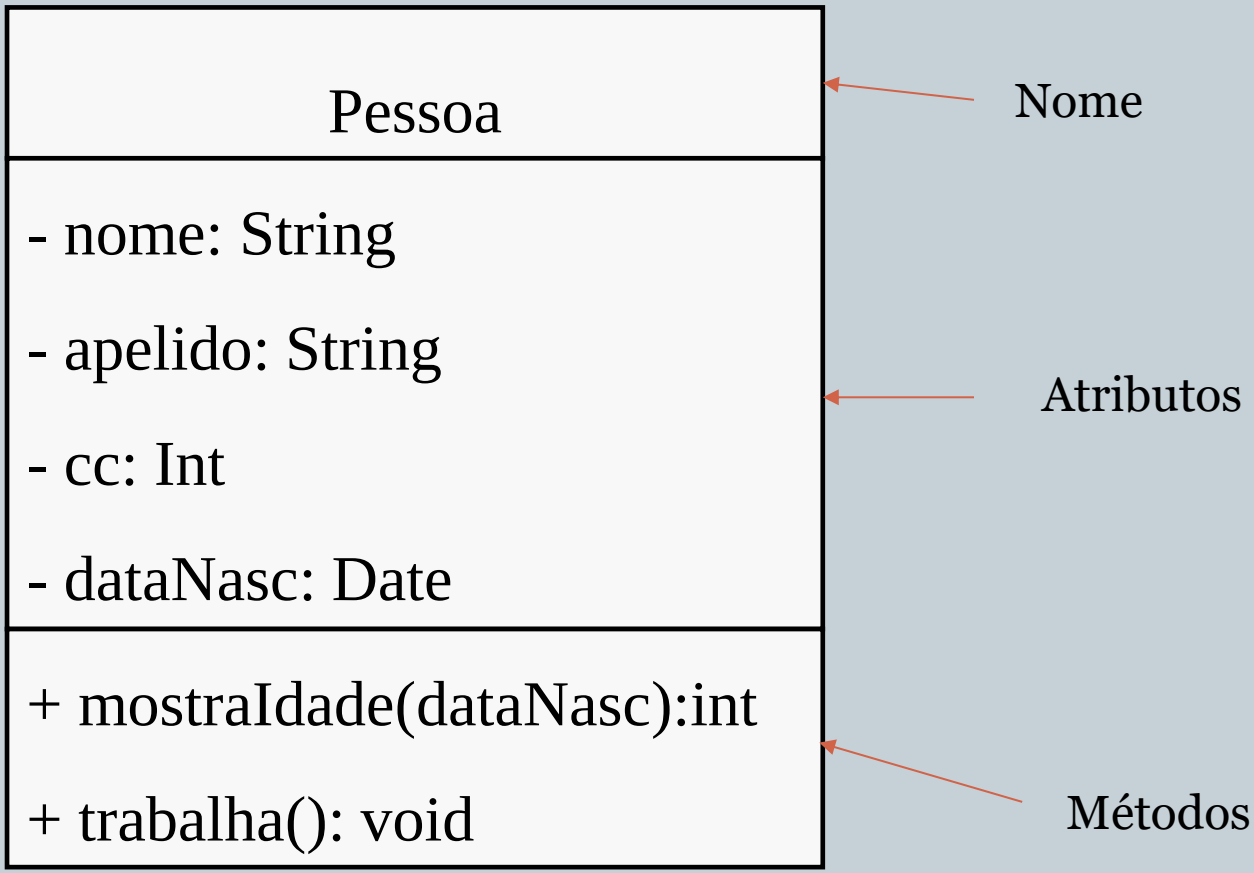
Cliente
nome: texto idade: número inteiro
criar(): void apagar(): void

Classes



- Visibilidade
  - A representação da visibilidade dos atributos e métodos das classes é feita através dos seguintes modificadores de acesso:
    - \+ Público - visível por qualquer outra classe
    - \# Protegido - visível na própria classe e suas subclasses (herança).
    - \- Privado - acessível apenas dentro da classe onde foi declarado
    - ~ Pacote - visível apenas por classes que pertencem ao mesmo pacote

Diagrama de Classes



Exemplos



- Classes
  - Pessoa - nome e idade;
  - Automóvel - marca e modelo
- Objetos
  - Pessoa:
    - Nome: Artur Silva, Idade: 33
    - Nome: Pedro Costa, Idade: 18
  - Automóvel:
    - Marca: Opel, Modelo: Corsa
    - Marca: Ford, Modelo: Fiesta

Identificação das classes



- Representam coisas, objetos, conceitos, eventos, dos quais queremos armazenar informação.
- A partir do enunciado do problema, procurar **substantivos** referenciados, ainda que de forma implícita.
- Cada substantivo ou cada “tema” presente deve ser examinado, em termos da importância de armazenamento de informação.

Identificação das classes



- Substantivos que produzam classes irrelevantes para o problema a resolver, devem ser eliminadas da escolha preliminar.
- Para classes redundantes, que surjam mais que uma vez com nomes diferentes, devem ser eliminadas as repetições.
- Escolher cuidadosamente os nomes para as classes. Se existirem diversos substantivos parecidos, escolher o que mais se adequa ao contexto do problema.

Identificação das classes

- Cada potencial classe dever ser examinada em termos das suas propriedades:
  - Se uma classe assegura a existência de uma entidade importante, deve ser mantida.
  - Se servir apenas para fornecer um valor, deverá ser transformada num atributo.
- A identificação de cada um dos mecanismo de interação entre o utilizador e o sistema deverá permitir se há classes em falta.

Identificação das classes



- Modelar não só as classes que tenham uma correspondência a objetos físicos, mas também a conceptuais que não tenham uma existência “tão palpável”.
- Se uma classe que tem uma propriedade que pode assumir um conjunto conhecido de valores, devemos modelar a classe através de um atributo ou de sub-classes?

Atributos ou Sub-classes ?

- A resposta é:
  - Se a cada valor da propriedade estão associadas características diferentes para os objetos, devemos usar uma sub-classe;
  - Se para os diferentes valores da propriedade a forma das instâncias não muda (temos os mesmos atributos, mas com valores diferentes) a propriedade deve ser representada através de um atributo.
  - Por exemplo, para representar o tipo de capa de um livro (capa dura, ou capa mole) num diagrama de classes, devemos fazê-lo através de um atributo.

Identificação de atributos nas classes

- Definida uma estrutura de classes, a definição do sistema a implementar deve ser cuidadosamente analisada, no sentido de identificar as propriedades de cada classe;

- Muitas vezes são listadas explicitamente (“cada venda terá um recibo com n.º, data e hora de emissão”).

- As frases possessivas e as enumerações são onde encontramos os atributos.

Identificação de atributos nas classes



- Manter apenas os atributos que são relevantes para a aplicação;
- Se existirem atributos discordantes numa dada classe, isso pode ser uma indicação de que necessitamos de outra classe ou de subclasses.
- Uma classe deve ser simples e coerente.

Fontes de informação



- Curso Prático de C#, FCA;
- C# 7.0 com Visual Studio, FCA;
- https://blog.betrybe.com/tecnologia/poo-programacao-orientada-a-objetos/
- https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264